

Проект: AI-агент службы поддержки

 15 августа 2026 г.

>_ НЕДОСТ...

 НЕДОСТ...

 В ПРОЦ...

 [Репозиторий](#)

 [Помощь](#)

-  Подготовка
- ✓ 1
- ✓ 2
- ✓ 3
- ✓ 4
- > 5
- > 6
- > 7
- > 8
- > 9
- > 10
-  Проверка

MCP ydb-tickets: Cloud Function + YDB

Подключаем к агенту собственный MCP-инструмент, который пишет обращения в YDB Serverless. Используем тот же паттерн Cloud Function + MCP Hub, но теперь инструмент не обращается во внешнюю систему, а работает с нашей собственной БД внутри Yandex Cloud.

Архитектурно это один Cloud Function `ydb-tickets`, который подключается к YDB через SDK для вашего языка (в примерах — Python-пакет `ydb`, PyPI-имя для `ydb-python-sdk`, см. <https://github.com/ydb-platform/ydb-python-sdk>) и предоставляет два инструмента через MCP Hub:

Инструмент	Что делает
<code>create-ticket</code>	Создаёт новый тикет
<code>list-my-tickets</code>	Возвращает тикеты пользователя по <code>user_id</code>

История переписки (таблица `messages`) — **не** MCP-инструмент: её пишет `email-poller` напрямую в YDB (см. ниже «Почему история — не MCP-инструмент»).

Схема YDB:

- 1 -- DROP'ы в начале — пере-инициализация схемы (данные теряются; для учебного проекта 
- 2 -- Если таблицы уже созданы и данные нужны — прокомментируйте DROP-блок.



Все курсы

```

6  CREATE TABLE tickets (
7      id            Utf8,          -- UUID
8      user_id       Utf8,          -- email отправителя
9      category      Utf8,          -- bug | docs | feature | access
10     status         Utf8,          -- open | answered | escalated | closed
11     text           Utf8,          -- текст обращения (после PII-маскирования)
12     created_at     Timestamp,
13     updated_at     Timestamp,
14     PRIMARY KEY (id),
15     INDEX tickets_by_user GLOBAL ON (user_id)  -- вторичный индекс для «мои заявки»
16 );
17
18 -- История переписки ведётся ПОЛЬЗОВАТЕЛЕМ (PK по user_id), а не тикетом:
19 -- почтовый диалог начинается до создания тикета (вопрос → RAG-ответ → «создай тикет»).
20 CREATE TABLE messages (
21     id            Utf8,          -- UUID
22     user_id       Utf8,          -- email отправителя / TG chat id
23     ticket_id     Utf8?,         -- ссылка на tickets.id; NULL, если реплика вне тикета
24     role          Utf8,          -- user | agent
25     text          Utf8,          -- текст сообщения (после PII-маскирования)
26     model         Utf8?,         -- какая модель отвечала (если role=agent) – заполняет poller
27     tokens_in     UInt64,        -- из usage ответа Responses API – пишет poller
28     tokens_out    UInt64,
29     latency_ms    UInt32,        -- замер poller'a
30     created_at     Timestamp,
31     PRIMARY KEY (user_id, id)
32 );

```

Почему история — не MCP-инструмент

У таблицы `messages` есть поля, которые LLM заполнить не может в принципе: `model`, `tokens_in/out`, `latency_ms`. Модель не знает, под каким именем её вызвали (это конфиг `poller'a`), не видит поле `usage` собственного ответа и не может замерить `latency`. Если попросить агента передавать эти данные через аргументы инструмента — в базу попадут выдуманные числа.

Разделение ответственности:

- **MCP-инструменты агента** — действия, где нужно ИИ-решение: завести тикет (`create-ticket`), показать прошлые заявки (`list-my-tickets`).
- **Код poller'a** — механика и телеметрия: `poller` пишет в `messages` входящее письмо (до вызова LLM) и ответ агента (после — с `model`, токенами из `usage` и замером `latency`).



Все курсы

`assistant` — агент «помнит» диалог между письмами.

- **Связка с тикетом:** если в том же ответе агент вызвал `create-ticket`, `poller` достаёт `ticket_id` из `mcp_call.output` и проставляет его обеим репликам цикла (`UPDATE`) — детерминированно, без участия LLM.

Ссылки

- [Yandex Managed Service for YDB](#) — документация от Яндекс
- [ydb-python-sdk](#) — официальный Python-клиент для YDB — репозиторий на GitHub
- [MCP Hub в Yandex AI Studio](#) — документация Yandex AI Studio
- [Responses API: создание ответа](#) — справочник API Yandex AI Studio

Задачи

1. Создайте таблицы в YDB. Два рабочих пути:

- UI: Yandex Cloud console → YDB → help-desk-db → Query → выполнить SQL из `src/ydb_tickets/schema.sql`.
- Python SDK локально: подключается через `ydb.AccessTokenCredentials(<iam>)`, читает `schema.sql`, выполняет по одному `statement`'у через `session.execute_scheme()`. Скрипт удаляет только строки, целиком начинающиеся с `--`; инлайновые комментарии внутри `CREATE TABLE` остаются (YQL их допускает).

```
1 source .venv/bin/activate
2 export YDB_ENDPOINT=grpc://ydb.serverless.yandexcloud.net:2135
3 export YDB_DATABASE=/ru-central1/<cloud-id>/<db-id>
4 export YC_IAM_TOKEN=$(~/yandex-cloud/bin/yc iam create-token)
5 python scripts/init_schema.py src/ydb_tickets/schema.sql
```



Схема уже менялась — `DROP`'ы в начале `schema.sql` пересоздают таблицы с нуля (данные теряются).

2. Реализуйте Cloud Function `ydb-tickets` на выбранном вами языке (в примерах — Python 3.12, `pip install ydb`, файл `src/ydb_tickets/index.py`). На вход — JSON. На выходе — JSON-ответ по контракту:

- `create-ticket` → `{"ticket_id": "...", "created_at": "..."} .`
- `list-my-tickets` → `[{"id": "...", "status": "...", "category": "...", "text": "...", "created_at": "..."}, ...] .`
- Этим `action`'ам соответствуют две записи в `mcp-tools.yaml`.



Все курсы

- Прямой invoke (`yc serverless function invoke`) — `event = {"action": "create-ticket", "user_id": "...", ...}`.
- HTTP через API Gateway — `event = {"httpMethod": "POST", "body": "<JSON-string>", ...}`.
- Вызов из MCP Hub — аргументы инструмента приходят прямо как `event` (без обёртки); диспетчеризация по набору ключей (см. «Подсказки»).

4. Задеплойте функцию. Формат `--secret` — `property=value` через запятую (`environment-variable`, `name` / `id`, `key`):

```
1  yc serverless function version create \
2    --function-name ydb-tickets \
3    --runtime python312 \
4    --entrypoint index.handle \
5    --memory 256m \
6    --execution-timeout 30s \
7    --source-path /tmp/ydb-tickets.zip \
8    --service-account-id <SA_ID> \
9    --secret environment-variable=YDB_ENDPOINT,name=ydb-endpoint,key=value \
10   --secret environment-variable=YDB_DATABASE,name=ydb-database,key=value
```



Упакуйте в zip: `zip -j /tmp/ydb-tickets.zip src/ydb_tickets/{index.py,requirements.txt}`.

Команда деплоя и состав архива — для Python-рантайма; под другой язык поменяйте `--runtime`, `--entrypoint` и содержимое zip.

5. Создайте MCP Hub gateway через CLI (а не только через UI):

```
1  yc serverless mcp-gateway create \
2    --name ydb-tickets-mcp \
3    --service-account-id <SA_ID> \
4    --tools-file src/ydb_tickets/mcp-tools.yaml
```



Структура `mcp-tools.yaml` — список из 2 инструментов. Важно: поле `input_json_schema` должно быть строкой (JSON-encoded), не YAML-объектом. Пример:

```
1  - name: create-ticket
2    description: "Создать тикет"
3    input_json_schema: '{"type":"object","properties":{"user_id":{"type":"string"}, ...}'
4    action:
5      function_call:
6
7
```





<CF_ID> — ID функции `ydb-tickets`: `yc serverless function get --name ydb-tickets --format json | jq -r .id`. Оба tool'a указывают на ту же CF `ydb-tickets`; диспетчеризация по ключам аргументов делается внутри функции.

6. Подключите MCP к агенту. В текущей версии AI Studio SA к агенту не привязывается — как это устроено, разбирается в шаге про настройку AI Studio (раздел «Как работает авторизация»). Вместо этого MCP-инструменты передаются inline в запросе Responses API или через UI MCP Hub (если используете сохранённого агента в Workflows).
7. В Responses API можно указать `require_approval: "never"` для MCP tool — иначе агент вернёт `mcp_approval_request` вместо `mcp_call`, и тикет не создастся:

```
1  "tools": [{
2      "type": "mcp",
3      "server_label": "ydb-tickets",
4      "server_url": "https://<mcpgw-id>.<salt>.mcpgw.serverless.yandexcloud.net/sse",
5      "require_approval": "never",
6  }]
```

Полный SSE-URL шлюза (`<mcpgw-id>.<salt>`) возьмите из вывода `yc serverless mcp-gateway create` или позже: `yc serverless mcp-gateway get --name ydb-tickets-mcp`. В массиве `output[]` ответа эти два типа различаются так: `mcp_call` — агент уже выполнил tool (есть `arguments` и `output`), а `mcp_approval_request` — агент только просит разрешение (есть `approval_request_id`, сам MCP-сервер не вызывался). С `require_approval: "never"` вы видите только `mcp_call`; с default — нужно вторым запросом дослать `mcp_approval_response` с `approve: true/false`, чтобы turn продолжился.

8. Доработайте email-roller: история переписки и телеметрия пишутся в YDB напрямую из roller'a (см. «Почему история — не MCP-инструмент»):

- Выдайте SA roller'a роль на запись в YDB: `yc resource-manager folder add-access-binding <folder-id> --role ydb.editor --service-account-id <SA_ID>`.
- В roller добавьте `import ydb` (упаковка — zip с `requirements.txt`, как у `ydb-tickets`) и переменные `YDB_ENDPOINT` / `YDB_DATABASE` (те же секреты Lockbox `ydb-endpoint` / `ydb-database`).
- Цикл обработки письма: (а) прочитать из `messages` последние реплики пользователя (`ORDER BY created_at DESC LIMIT N`, в обратном порядке — от старых к новым); (б) записать входящее письмо (`role=user`, PII-маскирование, `ticket_id=NULL`) — до вызова LLM; (в) вызвать Responses API, передав историю в `input` (`role agent` → `assistant`) и текущее письмо последним сообщением; (г) записать ответ (`role=agent`, `model` из конфига roller'a, `tokens_in/out` из `usage`, `latency_ms` — замер `time.monotonic()` вокруг вызова).



9. Обновите инструкции агента — опишите, когда агент должен вызывать каждый из двух инструментов: когда заводить тикет (`create-ticket`), когда показывать пользователю его ранее созданные заявки (`list-my-tickets`). Про сохранение истории в инструкциях писать не нужно — этим занимается `poller`.
10. Проверка end-to-end через почту (pull-режим из шага про `email-workflow`): «создай тикет категория `bug` текст: ...» → в логах `poller`'а видно `mcp_call name=create-ticket`, в YDB появляются записи в `tickets` и `messages` (реплики с проставленным `ticket_id`).

Подсказки

- Таблица `messages` нужна для разбора и учёта токенов, а не для памяти агента: читать её обратно не требуется. В почте историю приносит сам клиент — отвечая, сотрудник цитирует переписку в теле письма, и агент видит контекст как часть обращения.
- Параметры в `ydb-python-sdk` работают только через `prepared statements`:
`session.prepare(yql) → transaction().execute(prepared, {"$name": value})`, ключи — с префиксом `$`.
- MCP Hub передаёт аргументы инструмента напрямую как `event` (без обёртки `{"tool": ...}`) — диспетчеризуйте `action` по набору ключей, иначе функция вернёт `unknown action: None`.
- Если тикет не создаётся — проверьте в логах CF, что секреты подцепились (`YDB_ENDPOINT` не пустой) и что в запросе стоит `require_approval: "never"`.
- `usage` лежит в корне ответа `Responses API`: `data["usage"]["input_tokens"] / ["output_tokens"]`.
- PII-маскирование текста перед записью в `messages` делайте в `poller`'е (той же логикой, что в CF `ydb-tickets`) — история пишется в обход CF.

Результат шага

- В YDB созданы таблицы `tickets` и `messages` (PK `messages` — `user_id + id`, `ticket_id` Nullable).
- Cloud Function `ydb-tickets` развёрнута и отвечает корректным JSON на оба `action`.
- MCP-шлюз `ydb-tickets-mcp` создан через `yc serverless mcp-gateway create --tools-file`, к нему подключены 2 инструмента.
- Email-poller пишет историю переписки: при обращении по почте агент вызывает `create-ticket` (в логах `mcp_call name=create-ticket args={...}`), а в `messages` появляются обе реплики цикла — входящая и ответ агента с заполненными `model`, `tokens_in/out` (из `usage`), `latency_ms`; при созданном тикете у обеих реплик проставлен `ticket_id`.
- Повторное письмо из того же диалога: в логах `poller`'а в `REQUEST_PAYLOAD` видна история переписки в `input` (`multi-turn`).

В

I

Н

99

</>

📄

🔗

🔗

🖼️

☰

🗪

🔗

👁️

 **Отправить**

Комментарии

- Альгис Швиторайтис**

21 сентября 2026 г., 22:57

1. Создана почта helpdesk_hexlet@cif-raz.ru и при запросе с нее приходит ответ

2. Исходное письмо в ящике помечено как прочитанное.

3. Можно проверить цепочку командой `yc serverless function logs email-poller --limit 30`
- Альгис Швиторайтис**

18 сентября 2026 г., 17:14

Секреты добавлены в Lockbox Сервисный аккаунт создан, права настроены Аген helpdesk создан Readme заполнен

Хекслет

- О нас
- Отзывы
- Справка
- Блог
- Вопросы и ответы
- Глоссарий

Направления

- Программирование
- ИИ
- Аналитика
- DevOps
- Тестирование
- Фронтенд